Arun Kumar 09-05-2026

SOFTWARE DESIGN DOCUMENT

Hotel Management System

Version: 1.0

Date: May 2026

Language: Java 17+

Architecture: Layered CLI Application

Test Framework: JUnit 5

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the internal architecture, class design, data structures, and interaction flows of the Hotel Management System (HMS). It is intended to guide developers in understanding how the system is structured and how its components collaborate.

1.2 Design Goals

- Separation of concerns — CLI layer is decoupled from business logic services.
- Interface-driven design — each service implements a corresponding interface.
- Fail-fast validation — typed custom exceptions are thrown at the service layer.
- Testability — helper methods expose internal state for JUnit 5 test cases.
- Simplicity — in-memory ArrayList storage; no ORM or database dependencies.

2. System Architecture

2.1 Architectural Style

The HMS follows a Layered Architecture with three distinct layers: Presentation (CLI), Business Logic (Services), and Data (Models). This keeps user-interaction code separate from domain logic.

Layer	Package	Responsibility
Presentation	com.hotel.cli	CommandHandler.java — parses user input, routes to services, displays output
Business Logic	com.hotel.services	Five service classes — implement business rules and operations
Contracts	com.hotel.interfaces	Five interfaces — define the API each service must honour
Data / Domain	com.hotel.models	Five model classes — plain data objects (Room, Booking, etc.)
Validation	com.hotel.utils	InputValidator — reusable validation utility
Error Handling	com.hotel.exceptions	Seven custom exception classes

2.2 Package Structure

```
hotel-booking/src/main/java/
  com/hotel/
    cli/          CommandHandler.java
    interfaces/   BookingOperations  CommissionOperations
RecommendationOperations
                  ReferralOperations  RoomOperations
    models/       Room  Booking  AlternativeHotel  ReferralRequest
CommissionBill
    services/     RoomService  BookingService  RecommendationService
                  ReferralService  CommissionService
    exceptions/   RoomNotFoundException  RoomAlreadyExistsException
RoomNotAvailableException
                  BookingNotFoundException  InvalidInputException
                  InvalidDateException  HotelNotFoundException
    utils/        InputValidator

hotel-booking/src/test/java/
  com/hotel/
    RoomServiceTest  BookingServiceTest  CommissionServiceTest
    ReferralServiceTest  RecommendationServiceTest  InputValidatorTest
```

3. Class Design

3.1 Model Classes (com.hotel.models)

Room

Field	Type	Default	Description
roomNumber	int	—	Unique room identifier
roomType	String	—	Single / Double / Deluxe / Suite
pricePerNight	double	—	Nightly rate in Rs.
isAvailable	boolean	true	Availability flag (not used in date-overlap booking logic)

Booking

Field	Type	Default	Description
bookingId	String	—	Auto-generated BK-XXX
roomNumber	int	—	References booked room
guestName	String	—	Full guest name (multi-word supported)
checkInDate	LocalDate	—	Parsed from DD-MM-YYYY input
checkOutDate	LocalDate	—	Must be after checkInDate
status	String	CONFIRMED	CONFIRMED or CANCELLED

AlternativeHotel

Field	Type	Default	Description
hotelId	String	—	e.g. KH001, AH001, PH001
hotelName	String	—	Display name
location	String	—	Area / street
contactNumber	String	—	Phone number string
customersReferred	int	0	Incremented on each referral; drives commission
commissionPerCustomer	double	—	Fixed rate per referral
availableRooms	ArrayList<Room>	[]	Pre-loaded rooms shown during recommendation

ReferralRequest

Field	Type	Default	Description
referralId	String	—	Auto-generated REF-XXX
customerName	String	—	Guest who was referred
hotelId	String	—	Target partner hotel ID
hotelName	String	—	Target partner hotel name

referralDate	LocalDate	now()	Date referral was created
status	String	PENDING	PENDING or CONFIRMED

CommissionBill

Field	Type	Default	Description
billId	String	—	Auto-generated BILL-XXX; preserved on update
hotelId	String	—	Partner hotel ID
hotelName	String	—	Partner hotel name
totalCustomersReferred	int	—	Snapshot of customersReferred at bill time
commissionPerCustomer	double	—	Rate at bill time
totalBillAmount	double	auto	totalCustomersReferred × commissionPerCustomer
billStatus	String	UNPAID	UNPAID or PAID
billGeneratedDate	LocalDate	now()	Date bill was generated

3.2 Interface Contracts (com.hotel.interfaces)

Interface	Implemented By	Key Methods
RoomOperations	RoomService	addRoom, deleteRoom, getRoom, getAllRooms, getAvailableRooms, isRoomAvailable, displayAllRooms, displayAvailableRooms
BookingOperations	BookingService	bookRoom, cancelBooking, getBooking, getAllBookings, displayAllBookings, displayBooking
RecommendationOperations	RecommendationService	getRecommendedHotels, getHotel, displayRecommendedHotels, hasAvailableHotels
ReferralOperations	ReferralService	addReferral, getReferralsByHotel, getTotalReferrals, displayAllReferrals
CommissionOperations	CommissionService	generateBill, getBill, getAllBills, markBillAsPaid, displayBill, displayAllBills

3.3 Service Classes (com.hotel.services)

RoomService

Implements RoomOperations. Maintains ArrayList<Room> rooms.

Method	Logic Summary
addRoom(number, type, price)	Validates inputs; checks for duplicate room number; creates Room(available=true); adds to list
deleteRoom(number)	Calls getRoom() to locate; removes from list
getRoom(number)	Linear search; throws RoomNotFoundException if not found
getAvailableRooms()	Filters rooms list by isAvailable==true
isRoomAvailable(number)	Calls getRoom(); returns room.isAvailable()
getTotalRooms()	Returns rooms.size() — used by tests

BookingService

Implements BookingOperations. Maintains ArrayList<Booking> bookings. Depends on RoomService.

Method	Logic Summary
bookRoom(number, name, in, out)	Validates name → calls roomService.getRoom() → parses dates → checks checkIn not past → checkOut after checkIn → calls isRoomAvailableForDates() → creates Booking(CONFIRMED)
isRoomAvailableForDates()	Private. Iterates bookings; for same room and non-CANCELLED status checks: checkIn < existingOut AND checkOut > existingIn → returns false on overlap
cancelBooking(id)	Calls getBooking(); sets status to CANCELLED; idempotent for already-cancelled

getBooking(id)	Linear search on bookingId; throws BookingNotFoundException
getLastBookingId()	Returns last element's bookingId; null if empty — used by tests

RecommendationService

Implements RecommendationOperations. Pre-loads 3 AlternativeHotel objects in constructor via loadAlternativeHotels().

Pre-loaded Hotel	ID	Rooms	Commission
Kunal's Hotel	KH001	3 (Deluxe 2500, Suite 4000, Single 1200)	Rs.1000/customer
Arman's Hotel	AH001	2 (Double 1800, Suite 3500)	Rs.800/customer
Pine View Hotel	PH001	2 (Single 1000, Double 2000)	Rs.500/customer

ReferralService

Implements ReferralOperations. Maintains ArrayList<ReferralRequest> referrals.

Method	Logic Summary
addReferral(name, hotel)	Validates name and hotel not null; generates REF-XXX ID; creates ReferralRequest(PENDING); calls hotel.incrementCustomersReferred()
getReferralsByHotel(id)	Filters referrals list by matching hotelId; returns sub-list (empty if none)
confirmReferral(id)	Searches by referralId; sets status CONFIRMED; prints message if not found (no exception)

CommissionService

Implements CommissionOperations. Maintains ArrayList<CommissionBill> bills.

Method	Logic Summary
generateBill(hotel)	Null check → 0-referrals early return → search for existing bill by hotelId → if found: remove and recreate with same billId → else: create new BILL-XXX; adds to list
getBill(hotelId)	Linear search; throws HotelNotFoundException if not found
markBillAsPaid(id)	Calls getBill(); sets status PAID; idempotent for already-paid

4. Exception Design (com.hotel.exceptions)

Exception Class	Extends	Thrown When
RoomNotFoundException	Exception	getRoom(), deleteRoom(), isRoomAvailable() — room number not in list
RoomAlreadyExistsException	Exception	addRoom() — duplicate room number detected
RoomNotAvailableException	Exception	bookRoom() — date overlap with existing CONFIRMED booking
BookingNotFoundException	Exception	getBooking(), cancelBooking(), displayBooking() — booking ID not found
InvalidInputException	Exception	addRoom() for invalid type/price/number; bookRoom() for blank name; addReferral() for blank name
InvalidDateException	Exception	bookRoom() — bad format, past check-in, or check-out not after check-in
HotelNotFoundException	Exception	getHotel(), getBill(), generateBill(null), markBillAsPaid() — hotel not found

5. Input Validation Design (com.hotel.utils.InputValidator)

InputValidator is a stateless utility class. All methods return boolean. It is used by CommandHandler and referenced by test cases. Service classes perform their own inline validation and throw exceptions directly rather than delegating to InputValidator.

Method	Valid Condition	Invalid Cases
isValidInput(s)	Not null and not blank after trim	null, "", " "
isPositiveNumber(n)	n > 0	0, negative
isValidRoomNumber(n)	n > 0 (integer)	0, negative
isValidRoomType(s)	Exact match in [Single, Double, Deluxe, Suite]	Case mismatch, unknown type, null, blank
isValidDate(s)	Parses as DD-MM-YYYY AND not before today	Wrong format, past date, null, blank
isValidDateRange(i,o)	checkOut strictly after checkIn	checkOut <= checkIn, bad format
isValidGuestName(s)	Not blank, matches [a-zA-Z]+ after trim	Numbers, special chars, null, blank
isValidBookingId(s)	Matches regex BK-[0-9]{3}	Wrong prefix, wrong digit count, lowercase
isValidReferralId(s)	Matches regex REF-[0-9]{3}	Wrong prefix, digit count, lowercase
isValidPrice(p)	p > 0	0, negative
isValidHotelId(s)	Matches regex [A-Z]{2}[0-9]{3}	Lowercase, wrong lengths, hyphen present

6. CLI Design (com.hotel.cli.CommandHandler)

6.1 Command Routing

CommandHandler.start() runs an infinite loop reading from Scanner(System.in). Each line is split by space; parts[0] is lowercased and matched in a switch statement that delegates to private handle*() methods.

Command Token	Handler Method	Service Called
add-room	handleAddRoom()	roomService.addRoom()
delete-room	handleDeleteRoom()	roomService.deleteRoom()
list-rooms	— (inline)	roomService.displayAllRooms()
available-rooms	— (inline)	roomService.displayAvailableRooms()
book	handleBookRoom()	bookingService.bookRoom() → handleRoomNotAvailable()
cancel	handleCancelBooking()	bookingService.cancelBooking()
list-bookings	— (inline)	bookingService.displayAllBookings()
view-booking	handleViewBooking()	bookingService.displayBooking()
list-referrals	— (inline)	referralService.displayAllReferrals()
generate-commission	handleGenerateCommission()	recommendationService.getHotel() → commissionService.generateBill()
view-commission	handleViewCommission()	commissionService.displayBill()
view-all-commissions	— (inline)	commissionService.displayAllBills()
mark-paid	handleMarkCommissionPaid()	commissionService.markBillAsPaid()
admin123	displayHelp()	— prints help + opens AWT frame
exit	— (inline)	Disposes AWT frame; breaks loop

6.2 Guest Name Parsing (Multi-Word Support)

The book command accepts names with any number of words. The parser assigns parts[1] as room number, parts[length-2] as check-in, parts[length-1] as check-out, and joins parts[2 .. length-3] with spaces as the guest name.

```
Input:  book 101 Arun Kumar Sharma 10-05-2026 13-05-2026
parts[1]            = "101"         → roomNumber
parts[2..length-3]  = "Arun Kumar Sharma" → guestName
parts[length-2]     = "10-05-2026"  → checkInDate
parts[length-1]     = "13-05-2026"  → checkOutDate
```

6.3 Not-Available Flow

When bookRoom() throws RoomNotAvailableException, CommandHandler catches it and calls handleRoomNotAvailable(guestName). This checks if partner hotels exist, prompts the guest with [1] Yes / [0] No, calls recommendationService.displayRecommendedHotels(), then handleHotelSelection() which calls referralService.addReferral().

6.4 AWT Cheat-Sheet Frame

Property	Value
Class	java.awt.Frame (stored as cheatSheetFrame field)
Size	600 × 750 px
Layout	GridLayout(0, 1) — one column, auto rows
Background	RGB(30, 30, 30) — dark grey
Title colour	RGB(255, 215, 0) — gold, 16pt bold Arial
Command colour	RGB(255, 255, 255) — white, 13pt bold Arial
Example colour	RGB(180, 180, 180) — light grey, 12pt bold Arial
Always on top	true
Resizable	false
Lifecycle	Created on admin123; disposed on exit or window-close event

7. Key Interaction Flows

7.1 Successful Room Booking

- 1. User types: book 101 Arun Kumar 10-05-2026 13-05-2026
- 2. CommandHandler.handleBookRoom() parses parts → roomNumber=101, guestName="Arun Kumar", dates
- 3. BookingService.bookRoom() validates guest name (not blank)
- 4. RoomService.getRoom(101) verifies room exists (throws RoomNotFoundException if not)
- 5. Dates parsed with DateTimeFormatter("dd-MM-yyyy"); throws InvalidDateException on bad format
- 6. checkIn not before LocalDate.now(); checkOut after checkIn
- 7. isRoomAvailableForDates() iterates bookings; no overlap found
- 8. New Booking("BK-001", 101, "Arun Kumar", checkIn, checkOut, "CONFIRMED") added
- 9. Output: "Booking confirmed successfully. Your Booking ID is: BK-001"

7.2 Room Not Available — Referral Flow

- 1. bookRoom() finds date overlap → throws RoomNotAvailableException
- 2. CommandHandler catches → calls handleRoomNotAvailable(guestName)
- 3. Checks recommendationService.hasAvailableHotels() → true
- 4. Displays prompt [1] Yes / [0] No
- 5. Guest enters "1" → recommendationService.displayRecommendedHotels() shows 3 hotels
- 6. handleHotelSelection() prompts for hotel number (1-3) or 0 to decline
- 7. Guest selects → referralService.addReferral(guestName, selectedHotel)
- 8. REF-XXX created; hotel.incrementCustomersReferred() called
- 9. Output: "Referral recorded. Driver is waiting to drop you to <hotel>."

7.3 Commission Bill Generation Flow

- 1. User types: generate-commission KH001
- 2. CommandHandler calls recommendationService.getHotel("KH001") → AlternativeHotel object
- 3. Throws HotelNotFoundException if ID not found
- 4. commissionService.generateBill(hotel) invoked
- 5. If customersReferred == 0: prints message, returns null
- 6. Searches bills list for existing KH001 bill
- 7. If found: removes old bill, creates new CommissionBill with same billId and updated count
- 8. If not found: creates new bill with BILL-XXX ID and UNPAID status
- 9. totalBillAmount = customersReferred × commissionPerCustomer calculated in CommissionBill constructor

8. Data Flow and ID Generation

8.1 ID Generation Pattern

All IDs are generated using a session-scoped integer counter formatted with String.format():

Entity	Format	Example	Counter Field
Booking	BK-XXX	BK-001	bookingCounter (starts at 1)
Referral	REF-XXX	REF-001	referralCounter (starts at 1)
Commission Bill	BILL-XXX	BILL-001	billCounter (starts at 1)

Note: Counters are not reset during a session. Commission Bill IDs are preserved when a bill is updated.

8.2 Key Data Relationships

From	To	Relationship
Booking	Room	bookingId references roomNumber (integer FK by convention)
ReferralRequest	AlternativeHotel	hotelId string references AlternativeHotel.hotelId
CommissionBill	AlternativeHotel	hotelId and hotelName copied at bill generation time
AlternativeHotel	Room (list)	availableRooms pre-loaded list; displayed during recommendation
ReferralService	AlternativeHotel	addReferral() calls hotel.incrementCustomersReferred() directly
CommissionService	AlternativeHotel	reads hotel.getCustomersReferred() to compute bill amount

9. Test Design

9.1 Test Strategy

All tests use JUnit 5. Each test class has a @BeforeEach setUp() method that creates fresh service instances, ensuring full test isolation. Tests are grouped by method and cover: happy paths, boundary conditions, validation failures, and display smoke tests.

9.2 Test Coverage by Class

RoomServiceTest

Method Group	Tests	Scenarios
addRoom()	12	Success, default availability, details stored, duplicate, zero/negative number, zero/negative price, empty/null/whitespace type, multiple rooms
deleteRoom()	4	Success, room no longer exists, not found, only target deleted
getRoom()	2	Success, not found
getAvailableRooms()	3	All available, excludes unavailable, empty when all unavailable
isRoomAvailable()	3	True by default, false when marked, not found exception
getTotalRooms()	2	Zero initially, correct after add+delete
Display methods	3	Smoke tests — doNotThrow for displayAllRooms, displayAvailableRooms

BookingServiceTest

Method Group	Tests	Scenarios
bookRoom() — happy path	8	Success, ID format, first ID=BK-001, status=CONFIRMED, details stored, multi-word name, non-overlapping dates, incrementing IDs
bookRoom() — failures	9	Empty/null/whitespace name, non-existent room, invalid date format, past check-in, check-out before/same as check-in, overlapping dates, partial overlap
cancelBooking()	4	Success, idempotent (already cancelled), not found, dates freed after cancel
getBooking()	2	Success, not found
getAllBookings()	2	Empty initially, returns all
getTotalBookings()	2	Zero initially, cancelled bookings counted
getLastBookingId()	2	Null when empty, returns last created
Display methods	4	Smoke tests — all display methods and not-found exceptions

CommissionServiceTest

Method Group	Tests	Scenarios
generateBill() — happy	10	Success, BILL-001 format, amount=3×1000=3000, default UNPAID, details stored, date=today, two hotels two bills, update keeps same ID, updated amount
generateBill() — failures	3	Null hotel throws, 0 referrals returns null, 0 referrals no bill added
getBill()	3	Success, non-existent ID throws, before generating throws
getAllBills()	2	Empty initially, returns all after generating
markBillAsPaid()	3	Success, idempotent, not found throws
Formula verification	2	10×1000=10000, 5×800=4000
Display methods	4	Smoke tests

ReferralServiceTest

Method Group	Tests	Scenarios
addReferral() — happy	9	Success, REF-001 format, default PENDING, details, counter increment, multiple referrals, two hotels, incrementing IDs
addReferral() — failures	4	Empty/null/whitespace name, null hotel
getReferralsByHotel()	3	Correct subset, empty for no referrals, empty for unknown ID
getTotalReferrals()	2	Zero initially, correct count
confirmReferral()	2	Status changes to CONFIRMED, non-existent ID does not throw
Display methods	2	Smoke tests

RecommendationServiceTest

Method Group	Tests	Scenarios
Preloaded data	11	Count=3, each hotel exists by name, KH001 has 3 rooms, AH001/PH001 have 2 rooms each, commissions 1000/800/500, initial referrals=0
getRecommendedHotels()	3	Non-null, size=3, contains KH001
getHotel()	4	Valid ID, invalid throws, empty throws, case-sensitive throws
hasAvailableHotels()	1	Returns true when loaded
getTotalHotels()	1	Returns 3
Display methods	1	Smoke test

InputValidatorTest

Validator Method	Tests	Scenarios
isValidInput()	5	Valid string, null, empty, whitespace, string with spaces
isPositiveNumber()	4	Positive, zero, negative, small positive
isValidRoomNumber()	4	Positive, zero, negative, one

isValidRoomType()	10	All 4 valid types, invalid type, lowercase, null, empty, whitespace, leading/trailing spaces
isValidDate()	8	Future, today, past, YYYY-MM-DD, MM/DD/YYYY, null, empty, random string
isValidDateRange()	5	Out after in, out before in, same date, invalid format, one day apart
isValidGuestName()	10	Single, full, long name, with numbers, special chars, empty, null, whitespace, upper, lower
isValidBookingId()	8	BK-001, BK-999, wrong prefix, 2 digits, 4 digits, lowercase, null, empty
isValidReferralId()	7	REF-001, REF-999, wrong prefix, 2 digits, lowercase, null, empty
isValidPrice()	4	Positive, zero, negative, small positive
isValidHotelId()	10	KH001, AH001, PH001, lowercase, 1-letter prefix, 3-letter prefix, 2/4 digits, null, empty, hyphenated